



X-Ray 사진을 바탕으로 한 폐렴 환자 구분

☼ 상태	완료
📅 마감일	@11/24/2025
≡ 설명	스프린트 미션 #6

스프린트 미션 #6

X-Ray 사진을 바탕으로 한 폐렴 환자 구분

- 이번 미션에서는 흉부 X-Ray 사진을 바탕으로 폐렴 환자를 구분하는 작업을 수행합니다.
- 이번 미션의 목표는 X-Ray 사진을 입력으로 받아 폐렴 여부를 구분하는 분류 (Classification) 모델을 만드는 것입니다. 아래 데이터셋을 활용하여 다양한 이미지 전처리 및 증강 기법과 Transfer Learning과 Fine-Tuning 기법을 실험해보고, 모델의 성능을 평가해 보세요.
- 사용 데이터셋 : [데이터 링크](#)(Chest X-Ray Images (Pneumonia))

데이터 확인 및 전처리

1. 데이터 확인

1-1) 데이터 로드 / 경로 설정

```
import kagglehub

# Download latest version
```

```
path = kagglehub.dataset_download("paultimothymooney/chest-xray-pneumonia")
print("Dataset path:", path)
```

```
train_dir = os.path.join(path, "chest_xray", "train")
val_dir = os.path.join(path, "chest_xray", "val")
test_dir = os.path.join(path, "chest_xray", "test")
```

1-2) 데이터 시각화

```
transforms = v2.Compose(
    [
        v2.Resize((224, 224)),
        v2.ToImage(),
        v2.ToDtype(dtype=torch.float32, scale=True),
    ]
)

# 데이터셋 로드
train_dataset = datasets.ImageFolder(train_dir, transform=transforms)

# DataLoader 생성
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

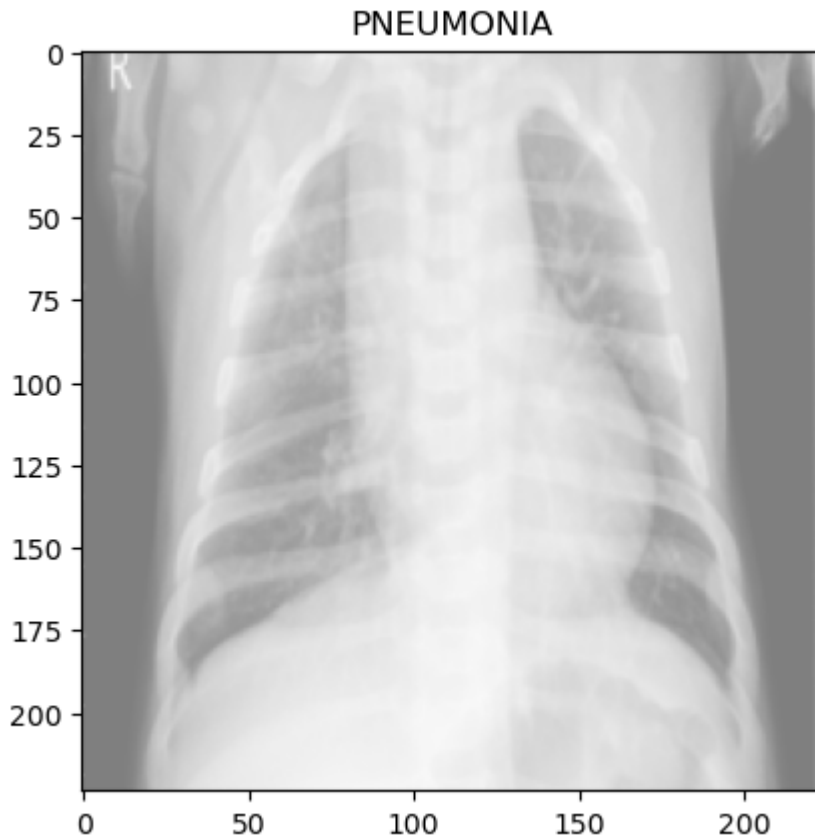
# 클래스 확인
class_names = train_dataset.classes
print("Class names:", class_names)
```

```
# 데이터 로더에서 배치 가져오기 (train)
images, labels = next(iter(train_loader))

# 이미지를 디스플레이
def imshow(img, title):
    img = img / 2 + 0.5 # 정규화 복원
    npimg = img.numpy()
```

```
plt.imshow(np.transpose(npimg, (1, 2, 0)))
plt.title(title)
plt.show()
```

```
# 첫 번째 배치의 첫 번째 이미지 시각화
imshow(images[0], title=class_names[labels[0]])
```



2. 데이터 전처리

- 이미지넷 전이 학습 모델을 구동할 예정이므로 이미지넷 평균과 표준편차로 정규화 진행.
- 훈련 데이터증강 → X-Ray 이미지에서 주로 좌우 반전은 허용

```
# 데이터 전처리 정의
```

```
imagenet_mean = [0.485, 0.456, 0.406] # 이미지넷 평균
```

```
imagenet_std = [0.229, 0.224, 0.225]
```

```
data_transforms = {
```

```

'train': v2.Compose([
    v2.Resize((224, 224)),          # 이미지 크기 조정
    v2.RandomHorizontalFlip(p=0.5), # 좌우 반전
    v2.ToTensor(),                  # 텐서로 변환
    v2.Normalize(mean=imagenet_mean, std=imagenet_std) # 정규
    화
]),
'val': v2.Compose([
    v2.Resize((224, 224)),          # 이미지 크기 조정
    v2.ToTensor(),                  # 텐서로 변환
    v2.Normalize(mean=imagenet_mean, std=imagenet_std) # 정규
    화
]),
'test': v2.Compose([
    v2.Resize((224, 224)),          # 이미지 크기 조정
    v2.ToTensor(),                  # 텐서로 변환
    v2.Normalize(mean=imagenet_mean, std=imagenet_std) # 정규
    화
])
}

```

데이터셋 로드

```

train_dataset = datasets.ImageFolder(train_dir, transform=data_transforms
['train'])
val_dataset = datasets.ImageFolder(val_dir, transform=data_transforms['va
l'])
test_dataset = datasets.ImageFolder(test_dir, transform=data_transforms['t
est'])

```

DataLoader 생성

```

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

```

```

print(f"Number of training samples: {len(train_dataset)}")
print(f"Number of validation samples: {len(val_dataset)}")
print(f"Number of test samples: {len(test_dataset)}")

```

전이학습 (DenseNet 121)

1. DenseNet 모델 정의

```
# 사전학습된 DenseNet121 불러오기
densenet = models.densenet121(weights=models.DenseNet121_Weights.IMAGENET1K_V1)

for param in densenet.features.parameters():
    param.requires_grad = False

num_features = densenet.classifier.in_features # 1024
densenet.classifier = nn.Linear(num_features, 2)

densenet = densenet.to(device)

# loss함수와 optim 설정

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(
    filter(lambda p: p.requires_grad, densenet.parameters()), lr=0.00001)
```

2. train, eval Loop 정의

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss = 0
    total_correct = 0
    total = 0

    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)
```

```

optimizer.zero_grad()
outputs = model(images) # [B, 2]

loss = criterion(outputs, labels)
loss.backward()
optimizer.step()

total_loss += loss.item() * images.size(0)

_, preds = torch.max(outputs, 1)
total_correct += (preds == labels).sum().item()
total += labels.size(0)

return total_loss / total, total_correct / total

def eval_one_epoch(model, loader, criterion, device):
    model.eval()
    total_loss = 0
    total_correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item() * images.size(0)
            _, preds = torch.max(outputs, 1)
            total_correct += (preds == labels).sum().item()
            total += labels.size(0)

    return total_loss / total, total_correct / total

```

3. 모델 학습 및 검증

3-1) 학습 루프

```
epochs = 10

for epoch in range(epochs):
    train_loss, train_acc = train_one_epoch(
        densenet, train_loader, criterion, optimizer, device)

    val_loss, val_acc = eval_one_epoch(
        densenet, val_loader, criterion, device)

    print(f"[{epoch+1}/{epochs}] "
          f"Train Loss: {train_loss:.4f}, Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Acc: {val_acc:.4f}")
```

3-2) 검증 루프

```
def test_model(loader, model, device):
    model.eval()
    all_preds = []
    all_labels = []

    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            # Accuracy 카운트
            correct += (preds == labels).sum().item()
```

```

total += labels.size(0)

# 전체 저장
all_preds.append(preds.cpu().numpy())
all_labels.append(labels.cpu().numpy())

# numpy로 변환
all_preds = np.concatenate(all_preds)
all_labels = np.concatenate(all_labels)

# Accuracy 계산
accuracy = correct / total

return all_labels, all_preds, accuracy

true_labels, pred_labels, test_acc = test_model(loader_model, test_loader,
device)
precision = precision_score(true_labels, pred_labels, average='binary')
recall    = recall_score(true_labels, pred_labels, average='binary')
f1        = f1_score(true_labels, pred_labels, average='binary')
cm = confusion_matrix(true_labels, pred_labels)

print(f"Test Accuracy: {test_acc:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-score  : {f1:.4f}")

```

4. DenseNet Partial Fine-tuning

4-1) Fine-tuning

```

# --- 1. 우선 전체 features는 다시 Freeze ---
for param in densenet.features.parameters():
    param.requires_grad = False

# --- 2. classifier(마지막 FC)는 계속 학습하게 둡니다 ---
for param in densenet.classifier.parameters():
    param.requires_grad = True

```



```

# --- 3. features 중에서도 "맨 끝 일부"만 풀기 ---
for name, param in densenet.features.named_parameters():
    # 마지막 두 레이어(denselayer15, denselayer16) + 최종 norm5만 풀기
    if ("denseblock4.denselayer15" in name or
        "denseblock4.denselayer16" in name or
        "norm5" in name):
        param.requires_grad = True
    # 디버깅용으로 어느 파라미터가 열렸는지 보고 싶으면:
    # print("Unfreeze:", name)

# Normal 쪽 recall을 조금 더 보정하기 위한 가중치 조정.
# Normal: index 0, PNEUMONIA: index 1

class_weights = torch.tensor([1.5, 1.0], device=device)
criterion = nn.CrossEntropyLoss(weight=class_weights)

# Normal을 잘못 맞추면 loss를 더 많이 먹음 →
# 모델이 Normal도 더 신경 쓰게 됨 → Normal recall 올라갈 가능성 ↑

trainable_params = filter(lambda p: p.requires_grad, densenet.parameters())

optimizer_ft = torch.optim.Adam(
    trainable_params,
    lr=1e-5,
    weight_decay=1e-4

```

4-2) 학습 루프

```

epochs_ft_2 = 10

for epoch in range(epochs_ft_2):
    train_loss, train_acc = train_one_epoch
                                (densenet, train_loader, criterion, optimizer_ft, device)

    val_loss, val_acc = eval_one_epoch

```

```
(densenet, val_loader, criterion, device)
```

```
print(f"[FT small {epoch+1}/{epochs_ft_2}] "  
      f"Train Loss: {train_loss:.4f}, Acc: {train_acc:.4f} | "  
      f"Val Loss: {val_loss:.4f}, Acc: {val_acc:.4f}")
```

4-3) 검증 루프

```
true_labels, pred_labels, test_acc = test_model(densenet, test_loader, device)
```

```
precision = precision_score(true_labels, pred_labels, average='binary')  
recall    = recall_score(true_labels, pred_labels, average='binary')  
f1        = f1_score(true_labels, pred_labels, average='binary')
```

```
print(f"Test Accuracy: {test_acc:.4f}")  
print(f"Precision : {precision:.4f}")  
print(f"Recall    : {recall:.4f}")  
print(f"F1-score  : {f1:.4f}")
```

5. DenseNet Full Fine-tuning

5-1) Full Fine-tuning

```
# DenseNet121 전체 레이어 풀기 (Full Fine-Tune)
```

```
for param in densenet.parameters():  
    param.requires_grad = True
```

```
# 손실함수
```

```
class_weights = torch.tensor([1.5, 1.0])
```

```
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
```

```
# 최적화
```

```
optimizer_full = torch.optim.Adam(  
    densenet.parameters(),  
    lr=1e-5,
```

```
weight_decay=1e-4
)
```

5-2) 학습 루프

```
epochs_full_ft = 10 # 3~5로 시작, 이후 더 늘릴지 판단

for epoch in range(epochs_full_ft):
    train_loss, train_acc = train_one_epoch(densenet, train_loader, criterion,
    optimizer_full, device)
    val_loss, val_acc = eval_one_epoch(densenet, val_loader, criterion, de
    vice)

    print(f"[Full FT {epoch+1}/{epochs_full_ft}] "
          f"Train Loss: {train_loss:.4f}, Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Acc: {val_acc:.4f}")
```

5-3) 검증 루프

```
true_labels, pred_labels, test_acc = test_model(densenet, test_loader, devic
e)

precision = precision_score(true_labels, pred_labels, average='binary')
recall = recall_score(true_labels, pred_labels, average='binary')
f1 = f1_score(true_labels, pred_labels, average='binary')
cm = confusion_matrix(true_labels, pred_labels)

print(f"Test Accuracy: {test_acc:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1-score : {f1:.4f}")
```

각 모델 별 시각화

1. Fine-tuning 별 시각화

▼ 시각화 코드

```
import matplotlib.pyplot as plt
import pandas as pd

# 1) 모델별 지표 정리
data = {
    "Model": [
        "Base (Freeze+FC)",
        "FT DenseBlock4",
        "FT Last Layers + CW",
        "Full FT"
    ],
    "Accuracy": [0.8478, 0.8173, 0.8526, 0.8413],
    "Precision": [0.8118, 0.7749, 0.8104, 0.7988],
    "Recall": [0.9846, 0.9974, 0.9974, 0.9974],
    "F1-score": [0.8899, 0.8722, 0.8943, 0.8871],
}

df = pd.DataFrame(data)
metrics = ["Accuracy", "Precision", "Recall", "F1-score"]

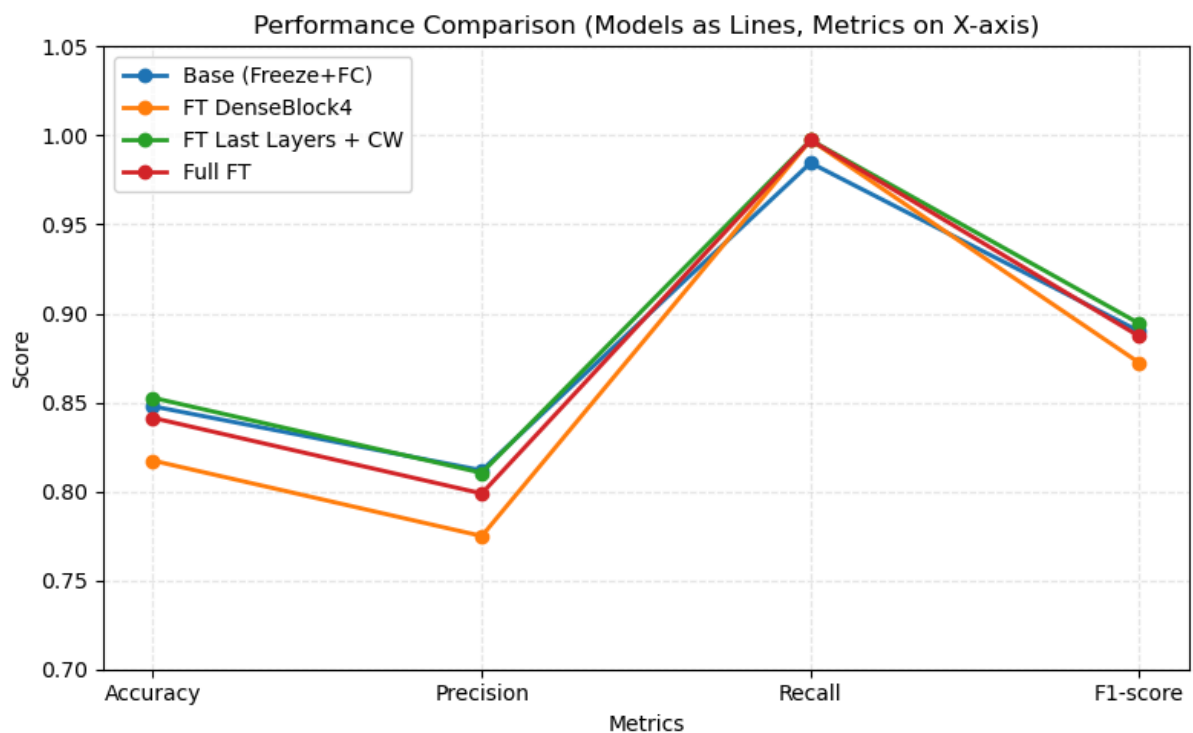
model_scores = {
    "Base (Freeze+FC)": [0.8478, 0.8118, 0.9846, 0.8899],
    "FT DenseBlock4": [0.8173, 0.7749, 0.9974, 0.8722],
    "FT Last Layers + CW": [0.8526, 0.8104, 0.9974, 0.8943],
    "Full FT": [0.8413, 0.7988, 0.9974, 0.8871],
}

plt.figure(figsize=(8, 5))

# 2) 각 모델이 하나의 라인이 되도록 plot
for model_name, scores in model_scores.items():
    plt.plot(metrics, scores, marker='o', linewidth=2, label=model_name)

# 3) 그래프 스타일
plt.ylim(0.7, 1.05)
plt.ylabel("Score")
```

```
plt.xlabel("Metrics")
plt.title("Performance Comparison (Models as Lines, Metrics on X-axis)")
plt.grid(linestyle="--", alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



- 보편적인 성능은 Full 보다 Partial이 더 균형있는 것으로 확인됨.

2. 이 외 전이학습 모델 별 시각화

▼ 시각화 코드

```
import matplotlib.pyplot as plt
import numpy as np
import json

with open("mission6_results.json", "r", encoding="utf-8") as f:
    data = json.load(f)
```

```

model_list = ["ResNet", "DenseNet", "EfficientNet", "MobileNet", "ViT"]
tuning_list = ["frozen", "partial", "full"]
score_list = ["accuracy", "precision", "recall", "f1"]

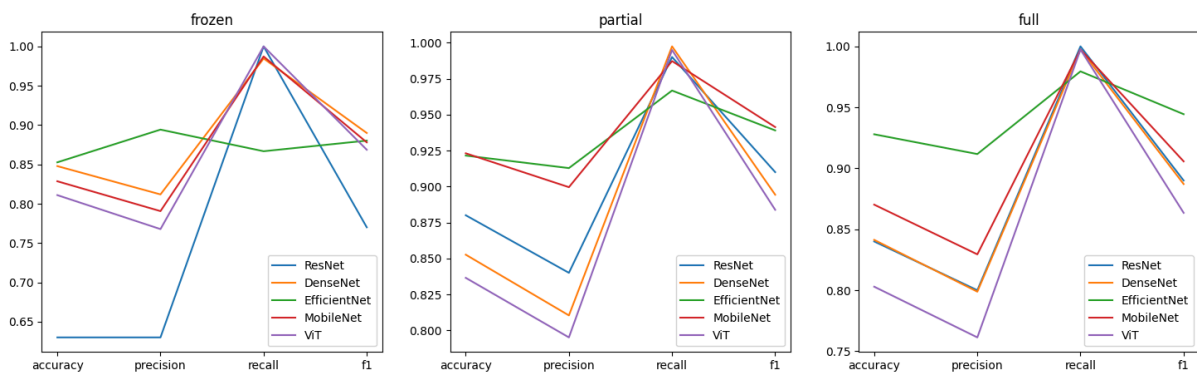
fig, axs=plt.subplots(1,3,figsize=(18,5))

for i, tuning in enumerate(tuning_list):

    for model in model_list:
        temp = []
        for score in score_list:
            temp.append(data[model][tuning][score])
        axs[i].plot(np.arange(0,4), temp, label=model)
    axs[i].set_title(tuning)
    axs[i].set_xticks([0,1,2,3], score_list)
    axs[i].legend(loc="lower right")

plt.show()

```



- 참고 자료.